

Homework 03: Blending and multiperiod planning

ISyE 524 — Fall 2026

Due: Monday, September 28, 2026, at 11:59 p.m. (Madison time).

Complete all three problems. For each problem, write a mathematical formulation before implementing it in Julia/JuMP. Define your variables, units, objective, constraints, and boundary conditions. Use continuous variables throughout this assignment.

The problems draw on Lectures 6–7: blending, inventory balances, staffing, and backlogging.

Getting started and written answers

Follow [Assignments and PDF Submission](#). In VS Code, use **Tasks: Run Task** to run **ISyE 524: Update course repository**, then **ISyE 524: Start an assignment from its template**, entering hw03. Open `student-work/hw03/hw03.ipynb` and select the Julia 1.12 kernel. An existing working copy will not be overwritten. Later course updates refresh the template in `assignments/hw03/`, but do not automatically update your personal assignment copy.

For **every written part**, type LaTeX mathematics in Markdown or insert a clear photograph/scan of handwritten work. Include the answer and reasoning together and label the subparts; no duplicate transcription is required. Julia/JuMP implementations belong in code cells.

Use `$x_1 \geq 0$` for inline mathematics and `$$x_1 + x_2 \leq 5$$` for display mathematics. Run the Markdown cell to render it.

For handwritten work, edit a Markdown cell in VS Code, drag in a cropped PNG/JPEG, and choose **Insert Image as Attachment**. Alternatively, put the image beside your working notebook under `student-work/hw03/images/` and insert `![My answer](images/problem1.png)`. Keep external images with the notebook. See [Images and handwritten work](#).

Back up `student-work/hw03/` separately; Git does not back up your answers or images. After completing the notebook, export it with the course PDF task, inspect `submissions/hw03.pdf` for readable mathematics, tables, outputs, and images, and submit that PDF to Gradescope.

Computational setup

Run the supplied setup and data cells in order. All data are included in this notebook. Use JuMP and HiGHS for every optimization model. Print the termination status and check that it is `MOI.OPTIMAL` and that a solution is available before reading values. Use unrounded values when checking constraints.

AI tools may help debug your own code, explain syntax or error messages, and critique your work. They should not replace constructing your formulation, implementation, or written solution. Complete the required collaboration and LLM-use statement at the end.

```
using JuMP, HiGHS, Printf
import MathOptInterface as MOI

# Prefer PNG if you add plots to your answers.
if isdefined(Main, :IJulia)
    filter!(mime -> mime != MIME("image/svg+xml"), IJulia.ijulia_mime_types)
end
```

1: Alloy Blending

Prof. Linderoth is planning on quitting his job as a professor to become an oligarch that controls the steel industry. But before I can do that, I need your help to optimize a specific order for steel that my company LOSE (Linderoth Optimal Steel Emporium) recently received.

I need to deliver an order for 500 tons of steel for the Naval shipyards in Norfolk. The steel must be composed of the following grades:

Element	Minimum (%)	Maximum (%)
Carbon (C)	2	3
Copper (Cu)	0.4	0.6
Manganese (Mn)	1.2	1.65

LOSE has seven different types of raw material in stock to complete this order. They each have different chemical composition and also cost me a specified amount per ton. There is also a maximum amount of each type available. This data is given below:

Raw	C (%)	Cu (%)	Mn (%)	Available (tons)	Cost (dollars/ton)
F	2.5	0	1.3	400	200
U	3	0.3	1.8	300	250
E	1.5	0.5	1.5	600	150
L	0	0.9	0	500	220
O	0.4	0.9	1.1	200	240
N	0	0.4	1.2	300	200
M	0	0.6	0	250	165

The order requires **at least 500 tons**. The customer permits excess delivery but pays no extra for it. All material blended is delivered. Quantities are continuous. Assume no material is lost in blending, and elemental composition is the mass-weighted average of the raw materials' compositions. Only the quantities actually used incur the listed costs.

1(a): Mathematical formulation

Write a linear programming instance that determines the number of tons of each raw that I need to put into my steel delivery to meet the grade requirements. My objective is to minimize the cost of the raw materials used to make the steel. Define variables and units; include the delivery requirement, raw-material availability limits, and both the lower and upper composition requirements for every element.

You may use indexed notation: J for raw materials, E for elements, c_j for cost, u_j for availability, γ_{ej} for the percentage of element e in raw j , α_e, β_e for minimum and maximum percentages, and $d = 500$ for the minimum delivery.

Starting with a weighted-average expression for an element's percentage, explain why you can rewrite each quality requirement as a linear constraint. Identify why multiplication by the denominator is valid here. Use a consistent convention for percentages: for example, 2 and 3, or 0.02 and 0.03, throughout.

Formulation and reasoning for 1(a):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

Sets: J for raw materials, E for elements

Parameters: c_j for cost, u_j for availability, γ_{ej} for the percentage of element e in raw j , α_e, β_e for minimum and maximum percentages, and $d = 500$ for the minimum delivery

$$\min \sum_{j \in J} c_j x_j$$

Subject To:

$$0 \leq x_j \leq u_j \quad \forall j \in J$$

$$\sum_{j \in J} x_j \geq d$$

$$\sum_{j \in J} x_j \alpha_e \leq \sum_{j \in J} \gamma_{ej} x_j \leq \sum_{j \in J} x_j \beta_e \quad \forall e \in E$$

You can write the requirements for an elements percentage because a weighted average is just a linear sum. This is because a weighted average is represented as $\bar{x} = \frac{\sum_{i=1}^n W_n x_n}{\sum_{i=1}^n W_n}$ and since this is a percentage the denominator is positive and can be multiplied out and the equation just becomes a linear summation of the weights as percentages. Thus the w_n in the numerator can be expressed as a minimum and maximum and set as the inequality

$$\sum_{j \in J} x_j \alpha_e \leq \sum_{j \in J} \gamma_{ej} x_j \leq \sum_{j \in J} x_j \beta_e \quad \forall e \in E$$

Supplied data for Problem 1

The following cell supplies the table values using the indexed notation above. In $\gamma[(e, j)]$, the element comes first and the raw material second.

```
J = ["F", "U", "E", "L", "O", "N", "M"]
E = ["C", "Cu", "Mn"]
c = Dict(zip(J, [200, 250, 150, 220, 240, 200, 165]))
u = Dict(zip(J, [400, 300, 600, 500, 200, 300, 250]))
α = Dict(zip(E, [2.0, 0.4, 1.2]))
β = Dict(zip(E, [3.0, 0.6, 1.65]))
d = 500.0

# Rows follow J; columns follow E. Entries are percentages.
composition = [
    2.5  0.0  1.3
    3.0  0.3  1.8
    1.5  0.5  1.5
    0.0  0.9  0.0
    0.4  0.9  1.1
    0.0  0.4  1.2
    0.0  0.6  0.0
]
γ = Dict{(e, j) => composition[k, l]}
    for (k, j) in enumerate(J), (l, e) in enumerate(E))
```

Dict{Tuple{String, String}, Float64} with 21 entries:

```
("Cu", "U") => 0.3
("Cu", "E") => 0.5
("C", "U") => 3.0
("C", "F") => 2.5
("C", "O") => 0.4
("Mn", "M") => 0.0
("Mn", "O") => 1.1
("Mn", "N") => 1.2
("C", "L") => 0.0
```

```

("Cu", "F") => 0.0
("Cu", "N") => 0.4
("Mn", "E") => 1.5
("Cu", "M") => 0.6
("Cu", "L") => 0.9
("C", "N") => 0.0
("Mn", "F") => 1.3
("C", "M") => 0.0
("C", "E") => 1.5
("Mn", "L") => 0.0
=>

```

1(b): Implement, solve, and check the blend

Implement your LP in JuMP and solve with HiGHS. Print the status, cost, tons of each raw used, and total tons delivered. Report the amount of each raw left unused.

Compute the percentage of C, Cu, and Mn in the resulting blend from the solved quantities. Compare each with both its minimum and maximum requirement. Which quality limits hold with equality? Check all availability limits and recompute the cost from the quantities.

```

# Build and solve the blending LP here.
# Check mass, availability, each quality bound, and cost.

# Build and solve the blending LP here.
# Check mass, availability, each quality bound, and cost.

using JuMP, HiGHS

lose=Model(HiGHS.Optimizer)

@variable(lose, x[J] >=0)

@constraint(lose, available[j in J],
    0<= sum(x[j]) <= u[j])

@constraint(lose, order,
    sum(x[j] for j in J) >= d)

@constraint(lose, makeup[e in E],
    α[e]sum(x[j] for j in J) <=
    sum(γ[e,j]x[j] for j in J))

@constraint(lose, makeup2[e in E],
    sum(γ[e,j]x[j] for j in J) <=
    β[e]sum(x[j] for j in J))

```

```

@Objective(lose, Min, sum(c[j]*x[j] for j in J))

optimize!(lose)
status = termination_status(lose)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(lose) || error("No feasible optimal solution is available")
println("Minimum Cost: ", objective_value(lose), " dollars")
solutions= Dict{() => value(x[j]) for j in J}
solution= Dict{() => value(x[j]) for j in J if value(x[j]) > 1e-6)

#Print Material Purchased\
println("")
for j in J
    if haskey(solution, (j))
        println(" Raw Mateial: ", j, " Amount Purchased: ", solution[j], " tons")
        println("Raw Material: ", j, " Amount Left Over: ", (u[j]-solution[j]),
    else
        println("Raw Material: ", j, " Amount Left Over: ", (u[j]), " tons")
    end
end

#Tons Delivered
delivered=sum(values(solution); init=0.0)
println("Amount of mateial delivered: ", delivered, " tons")

#Elements Used
println("")
for e in E
    element=sum(value(x[j])*γ[e,j] for j in J)/delivered
    println("Element: ", e, " Precentage: ", element)
end

#Print Cost
println("")
cost=sum(value(x[j])*c[j] for j in J; init=0.0)
println("Cost calculated from outputs: ", cost, " dollars")

```

Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT license
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full details
Using BLAS: blastrampoline
LP has 14 rows; 7 cols; 52 nonzeros
Coefficient ranges:

Matrix	[1e-01, 3e+00]
Cost	[2e+02, 2e+02]
Bound	[0e+00, 0e+00]
RHS	[2e+02, 6e+02]

```

Presolving model
6 rows, 7 cols, 39 nonzeros 0s
6 rows, 7 cols, 39 nonzeros 0s
Presolve reductions: rows 6(-8); columns 7(-0); nonzeros 39(-13)
Solving the presolved LP
Using dual simplex solver
  Iteration      Objective      Infeasibilities num(sum)
        0      0.0000000000e+00 Pr: 1(250) 0.0s
        3      9.0909090909e+04 Pr: 0(0) 0.0s

```

```

Performed postsolve
Solving the original LP from the solution after postsolve

```

```

Model status      : Optimal
Simplex iterations: 3
Objective value    : 9.0909090909e+04
P-D objective error: 0.0000000000e+00
HiGHS run time    : 0.00
Minimum Cost: 90909.09090909091 dollars

```

```

Raw Mateial: F Amount Purchased: 45.45454545454541 tons
Raw Material: F Amount Left Over: 354.5454545454546 tons
Raw Mateial: U Amount Purchased: 136.3636363636364 tons
Raw Material: U Amount Left Over: 163.6363636363636 tons
Raw Mateial: E Amount Purchased: 318.18181818181824 tons
Raw Material: E Amount Left Over: 281.81818181818176 tons
Raw Material: L Amount Left Over: 500 tons
Raw Material: O Amount Left Over: 200 tons
Raw Material: N Amount Left Over: 300 tons
Raw Material: M Amount Left Over: 250 tons
Amount of mateial delivered: 500.0 tons

```

```

Element: C Precentage: 2.0000000000000004
Element: Cu Precentage: 0.40000000000000013
Element: Mn Precentage: 1.5636363636363634

```

```

Cost calculated from outputs: 90909.09090909091 dollars

```

Results and interpretation for 1(b):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

The optimized soloution is to purchase 45.45 tons F, 136.36 tons U and 318.18 tons E. This will result in a minimum cost of 90909.09 dollars deleivering exactly 500 tons with a material makup of 2% C, 0.4% Cu and 1.5636% Mn with the values of C and Cu being at the minimum quality limit. There will be 250 tons of M, 300 tons N, 200 tons O, 500 tons L, 281.81 tons E, 163.63 tons U, and 354.54 tons F left over from the supplier after this purchase.

1(c): At least 500 versus exactly 500

Without another solver run, show that **every** minimum-cost solution to this instance delivers exactly 500 tons.

If a feasible blend delivers $T > 500$ tons, consider multiplying every quantity by $500/T$. Explain what happens to composition, availability, delivery, and cost. Which assumption about the costs makes the improvement strict?

Would replacing the delivery constraint by equality leave the entire feasible set unchanged, or only preserve the optimal decisions and value for this instance? Explain.

Reasoning for 1(c):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

Every cost minimum solution delivers exactly 500 tons because any values that result in a tonnage greater than 500 can be scaled down to 500. This is because the feasible region is bounded on the lower end only by the 500 value and the composition requirements otherwise all the bounds are on the upper end (direction of maximizing). This means that if you were to change the inequality to an equality you would only preserve the optimal decisions and values for this instance and remove much of the feasible region. |

2: So Much Grading!

The TA and I have *so much grading* to do. Over the next five months, we anticipate that we will need additional staffing to complete our grading efforts. During each of the next five months it is estimated that the number of hours of grading that is necessary is given in the table below.

Month	Grading (hours)
September	450
October	550
November	600
December	450
January	400

The UW pays 1,500 dollars per month per grader and each grader can handle as many as 16 hours of grading per month. We currently employ 30 graders. Graders can be hired only at the beginning of any month, and for each grader that is hired, the department incurs a one-time cost of 500 dollars. We can also ask a grader to retire (or we can fire them), for a one-time cost of 1,000 dollars per grader, at the beginning of a month.

Grading need not be completed in the month it arrives, i.e., some of it can be “carried over” to later months. However, it is bad to take too long for grading homeworks, so the department penalizes itself 1 dollar for each hour of ungraded homework at the end of the month. All grading must be done by the end of the fifth month.

Assume there is no unfinished grading before September. Work cannot be completed before it arrives. Graders hired at the beginning of a month can work that month, and graders who retire then receive no salary that month. Treat staffing as continuous grader equivalents for this LP; no rounding or integer restrictions are required. There is no prescribed final workforce size.

2(a): Mathematical formulation

Formulate a linear program whose solution will determine how many (if any) graders to hire at the beginning of each month and how many (if any) to ask to retire at the beginning of each month in order to minimize the total cost to the department over the next five months, including the penalty cost of not completing the work on time.

Define your variables and write the workforce and unfinished-work balances, capacity limits, initial conditions, and final requirement. You may use $T = \{1, \dots, 5\}$ with W_t denoting the grading hours that arrive in month t .

Formulation and reasoning for 2(a):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work.

For each month $t = 1, \dots, 5$, use:

Variable	Meaning	Units
g_t	Graders employed after hiring and firing	Graders
h_t	Graders hired at the start of the month	Graders
f_t	Graders fired at the start of the month	Graders
i_t	Backlog inventory to carry over at month end	Hours

All variables are nonnegative except, i_t which allows backlogging.

Objective Function

$$\min \sum_{t=1}^T 1500g_t + 1000f_t + 500h_t + 1i_t$$

Subject To:

$$\begin{aligned}g_0 &= 30 \\i_0 &= 0 \\h_t, f_t, g_t &\geq 0, t \in T \\i_{t-1} + 16g_t &= w_t + i_t, t \in T \\i_t &\geq 0, t \in T - 1 \\i_T &= 0 \\g_{t-1} + h_t - f_t &= g_t, t \in T \\T &= \{1, \dots, 5\}\end{aligned}$$

Supplied data for Problem 2

Use integer month indices in chronological order. Define your own decision-variable names in your formulation and implementation.

```
T = 1:5
month_name = ["September", "October", "November", "December", "January"]
W = [450, 550, 600, 450, 400]
InitialGraders = 30
HoursPerGrader = 16
Salary = 1500
HireCost = 500
RetireCost = 1000
BacklogCost = 1
```

1

2(b): Implement, solve, and interpret

Implement and solve your model from part 2(a) using Julia and JuMP. Be sure to show the number of graders hired and retired each month, and the number of hours of homework left ungraded at the end of each month.

Also report the status, minimum total cost, workforce size, and hours completed each month. Check the monthly balances and capacity limits and confirm that all grading is finished by January. Explain how the plan trades staffing costs against delayed grading, and interpret any fractional staffing values under the continuous assumption.

```
# Build and solve your staffing model here. Report and check the monthly plan.
using JuMP, HiGHS, DataFrames
```

```

grades=Model(HiGHS.Optimizer)

@variable(grades, gr[T] >=0) #graders
@variable(grades, f[T] >=0) #graders fired
@variable(grades, h[T] >=0) #graders hired
@variable(grades, i[T]>=0) #hours not fulfilled/backlog

@constraint(grades, init_hours, i[1]== W[1]-HoursPerGrader*gr[1])
@constraint(grades, hours[t in 2:length(T)],
    i[t]== W[t]+i[t-1]-HoursPerGrader*gr[t])

@constraint(grades, init_graders, InitialGraders + h[1]-f[1]==gr[1])
@constraint(grades, graders[t in 2:length(T)], gr[t-1]+h[t]-f[t]==gr[t])
@constraint(grades, clear_backlog, i[length(T)]==0)

@objective(grades, Min,
    sum(Salary*gr[t]+RetireCost*f[t]+HireCost*h[t]+BacklogCost*i[t] for t in T))

optimize!(grades)
status = termination_status(grades)
println("Termination status: ", status)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(grades) || error("No feasible optimal solution is available")

minimum_cost = objective_value(grades)
println("Minimum total cost: ", minimum_cost, " dollars")

plan = DataFrame(
    month = collect(month_name),
    hours_needed = W,
    graders_employed = value.(gr).data, #the dot is needed to extract all values
    graders_fired = value.(f).data,
    graders_hired = value.(h).data,
    hours_worked= value.(gr).data.*HoursPerGrader,
    hours_left = value.(i).data,
)

plan

```

Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT license
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full details
Using BLAS: blastrampoline
LP has 11 rows; 20 cols; 34 nonzeros

Coefficient ranges:

Matrix [1e+00, 2e+01]
Cost [1e+00, 2e+03]
Bound [0e+00, 0e+00]
RHS [3e+01, 6e+02]

Presolving model

8 rows, 16 cols, 27 nonzeros 0s

Dependent equations search running on 7 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.

7 rows, 15 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-4); columns 15(-5); nonzeros 25(-9)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
0	0.0000000000e+00	Ph1: 0(0) 0.0s
9	2.3342500000e+05	Pr: 0(0) 0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 9

Objective value : 2.3342500000e+05

P-D objective error : 0.0000000000e+00

HiGHS run time : 0.00

Termination status: OPTIMAL

Minimum total cost: 233425.0 dollars

	month	hours_needed	graders_employed	graders_fired	graders_hired	hours_worked	
	String	Int64	Float64	Float64	Float64	Float64	
1	September	450	28.125	1.875	0.0	450.0	...
2	October	550	31.25	0.0	3.125	500.0	...
3	November	600	31.25	0.0	0.0	500.0	...
4	December	450	31.25	0.0	0.0	500.0	...
5	January	400	31.25	0.0	0.0	500.0	...

Results and interpretation for 2(b):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work.

In this problem the lowest cost solution costs 233,425 dollars and requires you to fire 1.875 graders in september and hire 3.125 graders in October. This leaves 50, 150 and 100 hours of grading backloged in months October, november and december respectively. This plan rels on the fact that late homework hours only cost 1 dollar per hour whihc is much cheaper than hireing and fireing alot of workers at \$1000 per. This all rels on the fact that workers can be hired not as full time where they only work part of their 16 hrs at a rate equivelent to the amount of hours they work. If the payment is \$1500 regardless of hours then this would not make sence because you would be paying an extra 1500 dollars for 2 hours worth of work.

3: The Goblet of Wine

In this problem, at Dumbledore’s request, we will be producing goblets for drinking wine. There are six types of goblets $G = \{g_1, g_2, \dots, g_6\}$ that are produced in batches of 1000 goblets. We will help Dumbledore plan his production for the next 12 weeks. Production can be incomplete—that is, a batch can contain fewer than 1000 goblets. The demand, in batches (or thousands), for each type of goblet over the next 12 weeks is given below.

Goblet	Week 1	2	3	4	5	6
g_1	20	22	18	35	17	19
g_2	17	19	23	20	11	10
g_3	18	35	17	10	9	21
g_4	31	45	24	38	41	20
g_5	23	20	23	15	10	22
g_6	22	18	20	19	18	35

Goblet	Week 7	8	9	10	11	12
g_1	23	20	29	30	28	32
g_2	12	34	21	23	30	12
g_3	23	15	10	0	13	17
g_4	19	37	28	12	30	37
g_5	18	30	28	7	15	10
g_6	0	28	12	30	21	23

For each type of goblet, the initial stock is known, as well as the required final stock level in thousands. The positive final stock level is in order for Dumbledore to combat the evil “end of horizon” effects. Per batch for every goblet type, the production and storage costs (in monetary units known as galleons) are also known. House elves make the goblets on magical machines, and the required working time and machine time (in hours) is also given. Storing the goblets takes space, and the required storage space per batch (measured in number of trays) is also given.

Quantity	g_1	g_2	g_3	g_4	g_5	g_6
Production cost (galleons/batch)	100	80	110	90	200	140
Holding cost (galleons/batch/week)	25	28	25	27	10	20
Initial stock (batches)	50	20	0	15	0	10
Minimum final stock (batches)	10	10	10	10	10	10
Elf time (hours/batch)	3	3	3	2	4	4
Machine time (hours/batch)	2	1	4	8	11	9
Storage space (trays/batch)	4	5	5	6	4	9

There is a limited amount of elf time and machine time available per week. Specifically, the number of working hours is limited to 420 hours/week, and the machines have a weekly capacity of 800 hours. In Hogwarts, storage space for at most 1000 trays is available.

Initially, all demand must be met in the week it occurs. Charge holding costs on stock at the end of each week, including week 12, and apply the storage limit to these same end-of-week stocks. Final stock requirements are lower bounds. Fractional batches are allowed.

3(a): Mathematical formulation

Formulate a linear program to determine the different types of goblets that need to be produced in each period to minimize the total cost of production and storage. Use a general indexed formulation and define all variables and parameters. Include initial and final stock conditions and each shared resource limit.

Formulation and reasoning for 3(a):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work.

For each month $t = 1, \dots, 12$, use:

Variable	Meaning	Units
x_{gt}	Goblets made of g type in t month	Batch of 1000 Goblets
p_g	Production cost	galleons/batch
h_g	Holding Cost	galleons/batch
i_{gt}	Inventory of goblet type g in month t	Batch of 1000 Goblet
d_{gt}	Demand of goblet type g in month t	Batch of 1000 Goblet
e_g	elf time cost of each goblet type	hours/batch
m_g	machine time cost of each goblet type	hours/batch
s_g	storage space cost of each goblet type	trays/batch
w	elf total working hours limit	hours
c	machine capacity	hours
st	storage capacity	trays
IO_g	Initial Inventory of each goblet	trays
IF_{Final_g}	Final Inventory of each goblet	trays \

All variables are nonnegative

Objective Function

$$\min \sum_{g=1}^G \sum_{t=1}^T (p_g x_{gt} + h_g i_{gt})$$

Subject To:

$$\begin{aligned}
\sum_{g=1}^G (i_{gt} s_g) &\leq st) \quad \text{for } t \text{ in } [1, T] \\
\sum_{g=1}^G (x_{gt} e_g) &\leq w) \quad \text{for } t \text{ in } [1, T] \\
\sum_{g=1}^G (x_{gt} m_g) &\leq c) \quad \text{for } t \text{ in } [1, T] \\
x_{gt} + i_{g(t-1)} &= d_{gt} + i_{g,t} \quad \text{for } t \text{ in } [1, T], \forall g \in G \\
x_{gt}, p_g, h_g, i_{gt}, d_{gt}, e_g, m_g, s_g &\geq 0 \\
i_{g,0} = I0_g, i_{g,12} &= IFinal_g,
\end{aligned}$$

Supplied data for Problem 3

Rows of demand follow G; columns are weeks 1–12. The dictionary $d[(g, t)]$ gives the demand for goblet type g in week t . The code uses $I0$, $IFinal$, and $machine_time$ for the initial stock, final stock, and machine hours per batch. c , h , w , and s give production cost, holding cost, elf hours, and storage trays per batch, respectively.

```

G = ["g1", "g2", "g3", "g4", "g5", "g6"]
T = 1:12
demand = [
    20 22 18 35 17 19 23 20 29 30 28 32
    17 19 23 20 11 10 12 34 21 23 30 12
    18 35 17 10 9 21 23 15 10 0 13 17
    31 45 24 38 41 20 19 37 28 12 30 37
    23 20 23 15 10 22 18 30 28 7 15 10
    22 18 20 19 18 35 0 28 12 30 21 23
]
d = Dict{(g, t) => demand[i, t] for (i, g) in enumerate(G), t in T)
c = Dict(zip(G, [100, 80, 110, 90, 200, 140]))
h = Dict(zip(G, [25, 28, 25, 27, 10, 20]))
I0 = Dict(zip(G, [50, 20, 0, 15, 0, 10]))
IFinal = Dict{g => 10 for g in G}
w = Dict(zip(G, [3, 3, 3, 2, 4, 4]))
machine_time = Dict(zip(G, [2, 1, 4, 8, 11, 9]))
s = Dict(zip(G, [4, 5, 5, 6, 4, 9]))
H = 420
M = 800
S = 1000

```

1000

3(b): Implement and solve

Implement your model from part 3(a) in Julia/JuMP and solve it with HiGHS. Display the status and minimum cost. Report the optimal production quantities and end-of-week inventory for each goblet type, with a separate summary of the end-of-week inventory for type g_2 .

Check the stock balances, weekly elf and machine capacities, storage limits, and final stock requirements. Briefly explain why some goblets may need to be produced before the week in which they are demanded.

```
# Build and solve the goblet production model. Report and check the plan.

goblet=Model(HiGHS.Optimizer)

@variable(goblet, x[G, T] >=0)
@variable(goblet, i[G, T] >=0)

@constraint(goblet, storage_limit[t in T],
    sum(i[g, t]*s[g] for g in G) <= S)

@constraint(goblet, elf_limit[t in T],
    sum(x[g, t]*w[g] for g in G) <= H)

@constraint(goblet, machine_limit[t in T],
    sum(x[g, t]*machine_time[g] for g in G) <= M)

@constraint(goblet, initial_inventory[g in G],
    x[g, 1]+I0[g] == d[g, 1]+i[g, 1])
@constraint(goblet, production[g in G, t in 2:length(T)],
    x[g, t]+i[g, t-1] == d[g, t]+i[g, t])
@constraint(goblet, final[g in G],
    x[g, length(T)]+i[g, length(T)-1] >= d[g, length(T)]+IFinal[g])

@objective(goblet, Min,
    sum((c[g]*x[g, t]+h[g]*i[g, t] for g in G, t in T)))

optimize!(goblet)
status = termination_status(goblet)
println("Termination status: ", status)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(goblet) || error("No feasible optimal solution is available")

minimum_cost_goblets = objective_value(goblet)
println("Minimum total cost: ", minimum_cost_goblets, " galleons")
solution= Dict{(g,t) => value(x[g, t]) for g in G, t in T)

#Print Product and amount Produced
```

```

println("")
for g in G, t in T
    println("Product: ",g," Week: ",
            t," Units Produced: ",solution[(g,t)], " Batches")
end

#Print Inventory at end of week
println("")
for g in G, t in T
    println("Product: ", g, " Week: ",
            t," Inventory: ", value(i[g,t]), " Batches")
end

#Print Inventory for G2 at end of week
println("")
for t in T
    println("Product: ", "g2", " Week: ",
            t," Inventory: ", value(i["g2",t]), " Batches")
end

#Print Stock Balances
println("")
for t in T
    stock= sum(i[g,t]*s[g] for g in G)
    println("Storage room Used on week",
            t, ": ", value(stock), " trays")
end

#Print Elf Use
println("")
for t in T
    elf= sum(x[g,t]*w[g] for g in G)
    println("Elf Work Hours Used",t, ": ", value(elf), " hours")
end

#Print Machine Use
println("")
for t in T
    mech= sum(x[g,t]*machine_time[g] for g in G)
    println("Machine Hours Used",t, ": ", value(mech), " hours")
end

```

Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT license
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full details

Using BLAS: blastrampoline

LP has 114 rows; 144 cols; 438 nonzeros

Coefficient ranges:

Matrix [1e+00, 1e+01]

Cost [1e+01, 2e+02]

Bound [0e+00, 0e+00]

RHS [3e+00, 1e+03]

Presolving model

108 rows, 138 cols, 426 nonzeros 0s

Dependent equations search running on 63 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.

105 rows, 135 cols, 423 nonzeros 0s

Presolve reductions: rows 105(-9); columns 135(-9); nonzeros 423(-15)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
-----------	-----------	--------------------------

0	1.0734280382e+04	Pr: 69(4844) 0.0s
---	------------------	-------------------

130	1.8197581818e+05	Pr: 0(0) 0.0s
-----	------------------	---------------

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 130

Objective value : 1.8197581818e+05

P-D objective error : 7.9965983342e-17

HiGHS run time : 0.00

Termination status: OPTIMAL

Minimum total cost: 181975.81818181818 galleons

Product: g1 Week: 1 Units Produced: 0.0 Batches

Product: g1 Week: 2 Units Produced: 0.0 Batches

Product: g1 Week: 3 Units Produced: 10.0 Batches

Product: g1 Week: 4 Units Produced: 35.0 Batches

Product: g1 Week: 5 Units Produced: 17.0 Batches

Product: g1 Week: 6 Units Produced: 19.0 Batches

Product: g1 Week: 7 Units Produced: 23.0 Batches

Product: g1 Week: 8 Units Produced: 20.0 Batches

Product: g1 Week: 9 Units Produced: 33.0 Batches

Product: g1 Week: 10 Units Produced: 26.0 Batches

Product: g1 Week: 11 Units Produced: 28.0 Batches

Product: g1 Week: 12 Units Produced: 42.0 Batches

Product: g2 Week: 1 Units Produced: 0.0 Batches

Product: g2 Week: 2 Units Produced: 16.0 Batches

Product: g2 Week: 3 Units Produced: 23.0 Batches

Product: g2 Week: 4 Units Produced: 20.0 Batches

Product: g2 Week: 5 Units Produced: 11.0 Batches

Product: g2 Week: 6 Units Produced: 10.0 Batches

Product: g2 Week: 7 Units Produced: 12.0 Batches
 Product: g2 Week: 8 Units Produced: 34.0 Batches
 Product: g2 Week: 9 Units Produced: 21.0 Batches
 Product: g2 Week: 10 Units Produced: 23.0 Batches
 Product: g2 Week: 11 Units Produced: 30.0 Batches
 Product: g2 Week: 12 Units Produced: 22.0 Batches
 Product: g3 Week: 1 Units Produced: 18.0 Batches
 Product: g3 Week: 2 Units Produced: 35.0 Batches
 Product: g3 Week: 3 Units Produced: 17.0 Batches
 Product: g3 Week: 4 Units Produced: 10.0 Batches
 Product: g3 Week: 5 Units Produced: 9.0 Batches
 Product: g3 Week: 6 Units Produced: 21.0 Batches
 Product: g3 Week: 7 Units Produced: 23.0 Batches
 Product: g3 Week: 8 Units Produced: 15.0 Batches
 Product: g3 Week: 9 Units Produced: 10.0 Batches
 Product: g3 Week: 10 Units Produced: 0.0 Batches
 Product: g3 Week: 11 Units Produced: 13.0 Batches
 Product: g3 Week: 12 Units Produced: 27.0 Batches
 Product: g4 Week: 1 Units Produced: 16.0 Batches
 Product: g4 Week: 2 Units Produced: 45.0 Batches
 Product: g4 Week: 3 Units Produced: 24.0 Batches
 Product: g4 Week: 4 Units Produced: 38.0 Batches
 Product: g4 Week: 5 Units Produced: 41.0 Batches
 Product: g4 Week: 6 Units Produced: 20.0 Batches
 Product: g4 Week: 7 Units Produced: 19.0 Batches
 Product: g4 Week: 8 Units Produced: 37.0 Batches
 Product: g4 Week: 9 Units Produced: 28.0 Batches
 Product: g4 Week: 10 Units Produced: 12.0 Batches
 Product: g4 Week: 11 Units Produced: 30.0 Batches
 Product: g4 Week: 12 Units Produced: 47.0 Batches
 Product: g5 Week: 1 Units Produced: 31.909090909090907 Batches
 Product: g5 Week: 2 Units Produced: 11.090909090909092 Batches
 Product: g5 Week: 3 Units Produced: 23.0 Batches
 Product: g5 Week: 4 Units Produced: 15.363636363636363 Batches
 Product: g5 Week: 5 Units Produced: 20.818181818181817 Batches
 Product: g5 Week: 6 Units Produced: 17.545454545454547 Batches
 Product: g5 Week: 7 Units Produced: 45.27272727272727 Batches
 Product: g5 Week: 8 Units Produced: 6.75 Batches
 Product: g5 Week: 9 Units Produced: 31.0 Batches
 Product: g5 Week: 10 Units Produced: 28.25 Batches
 Product: g5 Week: 11 Units Produced: 0.0 Batches
 Product: g5 Week: 12 Units Produced: 0.0 Batches
 Product: g6 Week: 1 Units Produced: 12.0 Batches
 Product: g6 Week: 2 Units Produced: 18.0 Batches
 Product: g6 Week: 3 Units Produced: 20.0 Batches
 Product: g6 Week: 4 Units Produced: 19.0 Batches
 Product: g6 Week: 5 Units Produced: 18.0 Batches
 Product: g6 Week: 6 Units Produced: 35.0 Batches

Product: g6 Week: 7 Units Produced: 0.0 Batches
Product: g6 Week: 8 Units Produced: 28.0 Batches
Product: g6 Week: 9 Units Produced: 12.0 Batches
Product: g6 Week: 10 Units Produced: 34.0 Batches
Product: g6 Week: 11 Units Produced: 36.75 Batches
Product: g6 Week: 12 Units Produced: 13.25 Batches

Product: g1 Week: 1 Inventory: 30.0 Batches
Product: g1 Week: 2 Inventory: 8.0 Batches
Product: g1 Week: 3 Inventory: 0.0 Batches
Product: g1 Week: 4 Inventory: 0.0 Batches
Product: g1 Week: 5 Inventory: 0.0 Batches
Product: g1 Week: 6 Inventory: 0.0 Batches
Product: g1 Week: 7 Inventory: 0.0 Batches
Product: g1 Week: 8 Inventory: 0.0 Batches
Product: g1 Week: 9 Inventory: 4.0 Batches
Product: g1 Week: 10 Inventory: 0.0 Batches
Product: g1 Week: 11 Inventory: 0.0 Batches
Product: g1 Week: 12 Inventory: 10.0 Batches
Product: g2 Week: 1 Inventory: 3.0 Batches
Product: g2 Week: 2 Inventory: 0.0 Batches
Product: g2 Week: 3 Inventory: 0.0 Batches
Product: g2 Week: 4 Inventory: 0.0 Batches
Product: g2 Week: 5 Inventory: 0.0 Batches
Product: g2 Week: 6 Inventory: 0.0 Batches
Product: g2 Week: 7 Inventory: 0.0 Batches
Product: g2 Week: 8 Inventory: 0.0 Batches
Product: g2 Week: 9 Inventory: 0.0 Batches
Product: g2 Week: 10 Inventory: 0.0 Batches
Product: g2 Week: 11 Inventory: 0.0 Batches
Product: g2 Week: 12 Inventory: 10.0 Batches
Product: g3 Week: 1 Inventory: 0.0 Batches
Product: g3 Week: 2 Inventory: 0.0 Batches
Product: g3 Week: 3 Inventory: 0.0 Batches
Product: g3 Week: 4 Inventory: 0.0 Batches
Product: g3 Week: 5 Inventory: 0.0 Batches
Product: g3 Week: 6 Inventory: 0.0 Batches
Product: g3 Week: 7 Inventory: 0.0 Batches
Product: g3 Week: 8 Inventory: 0.0 Batches
Product: g3 Week: 9 Inventory: 0.0 Batches
Product: g3 Week: 10 Inventory: -0.0 Batches
Product: g3 Week: 11 Inventory: 0.0 Batches
Product: g3 Week: 12 Inventory: 10.0 Batches
Product: g4 Week: 1 Inventory: 0.0 Batches
Product: g4 Week: 2 Inventory: 0.0 Batches
Product: g4 Week: 3 Inventory: 0.0 Batches
Product: g4 Week: 4 Inventory: 0.0 Batches
Product: g4 Week: 5 Inventory: 0.0 Batches

Product: g4 Week: 6 Inventory: 0.0 Batches
 Product: g4 Week: 7 Inventory: 0.0 Batches
 Product: g4 Week: 8 Inventory: 0.0 Batches
 Product: g4 Week: 9 Inventory: 0.0 Batches
 Product: g4 Week: 10 Inventory: 0.0 Batches
 Product: g4 Week: 11 Inventory: 0.0 Batches
 Product: g4 Week: 12 Inventory: 10.0 Batches
 Product: g5 Week: 1 Inventory: 8.909090909090908 Batches
 Product: g5 Week: 2 Inventory: 0.0 Batches
 Product: g5 Week: 3 Inventory: 0.0 Batches
 Product: g5 Week: 4 Inventory: 0.3636363636363633 Batches
 Product: g5 Week: 5 Inventory: 11.18181818181818 Batches
 Product: g5 Week: 6 Inventory: 6.727272727272727 Batches
 Product: g5 Week: 7 Inventory: 34.0 Batches
 Product: g5 Week: 8 Inventory: 10.75 Batches
 Product: g5 Week: 9 Inventory: 13.75 Batches
 Product: g5 Week: 10 Inventory: 35.0 Batches
 Product: g5 Week: 11 Inventory: 20.0 Batches
 Product: g5 Week: 12 Inventory: 10.0 Batches
 Product: g6 Week: 1 Inventory: 0.0 Batches
 Product: g6 Week: 2 Inventory: 0.0 Batches
 Product: g6 Week: 3 Inventory: 0.0 Batches
 Product: g6 Week: 4 Inventory: 0.0 Batches
 Product: g6 Week: 5 Inventory: 0.0 Batches
 Product: g6 Week: 6 Inventory: 0.0 Batches
 Product: g6 Week: 7 Inventory: -0.0 Batches
 Product: g6 Week: 8 Inventory: 0.0 Batches
 Product: g6 Week: 9 Inventory: 0.0 Batches
 Product: g6 Week: 10 Inventory: 4.0 Batches
 Product: g6 Week: 11 Inventory: 19.75 Batches
 Product: g6 Week: 12 Inventory: 10.0 Batches

Product: g2 Week: 1 Inventory: 3.0 Batches
 Product: g2 Week: 2 Inventory: 0.0 Batches
 Product: g2 Week: 3 Inventory: 0.0 Batches
 Product: g2 Week: 4 Inventory: 0.0 Batches
 Product: g2 Week: 5 Inventory: 0.0 Batches
 Product: g2 Week: 6 Inventory: 0.0 Batches
 Product: g2 Week: 7 Inventory: 0.0 Batches
 Product: g2 Week: 8 Inventory: 0.0 Batches
 Product: g2 Week: 9 Inventory: 0.0 Batches
 Product: g2 Week: 10 Inventory: 0.0 Batches
 Product: g2 Week: 11 Inventory: 0.0 Batches
 Product: g2 Week: 12 Inventory: 10.0 Batches

Storage room Used on week1: 170.63636363636363 trays
 Storage room Used on week2: 32.0 trays
 Storage room Used on week3: 0.0 trays

Storage room Used on week4: 1.4545454545454533 trays
Storage room Used on week5: 44.72727272727272 trays
Storage room Used on week6: 26.909090909090907 trays
Storage room Used on week7: 136.0 trays
Storage room Used on week8: 43.0 trays
Storage room Used on week9: 71.0 trays
Storage room Used on week10: 176.0 trays
Storage room Used on week11: 257.75 trays
Storage room Used on week12: 330.0 trays

Elf Work Hours Used1: 261.6363636363636 hours
Elf Work Hours Used2: 359.3636363636364 hours
Elf Work Hours Used3: 370.0 hours
Elf Work Hours Used4: 408.45454545454544 hours
Elf Work Hours Used5: 348.27272727272725 hours
Elf Work Hours Used6: 400.1818181818182 hours
Elf Work Hours Used7: 393.0909090909091 hours
Elf Work Hours Used8: 420.0 hours
Elf Work Hours Used9: 420.0 hours
Elf Work Hours Used10: 420.0 hours
Elf Work Hours Used11: 420.0 hours
Elf Work Hours Used12: 420.0 hours

Machine Hours Used1: 659.0 hours
Machine Hours Used2: 800.0 hours
Machine Hours Used3: 736.0 hours
Machine Hours Used4: 774.0 hours
Machine Hours Used5: 800.0 hours
Machine Hours Used6: 800.0 hours
Machine Hours Used7: 800.0 hours
Machine Hours Used8: 756.25 hours
Machine Hours Used9: 800.0 hours
Machine Hours Used10: 787.75 hours
Machine Hours Used11: 708.75 hours
Machine Hours Used12: 709.25 hours

Results and interpretation for 3(b):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work.

The minimum cost for this goblet manufacturing senario is 181,975 galleons. During this production schedule you will be limited in production many weeks by either machine hours or elf hours so to meet demand dubmledore needs to overproduce many weeks to make up for future weeks running out of machine or elf hours.

3(c): Allow backlogging

Determine how the production schedule changes if goblet demand is allowed to be backlogged at **zero** cost. The final inventory requirements must still be met, and all demand must have been filled by the end of week 12. All other data remain as in parts 3(a)–3(b). Holding costs and tray usage apply only to physical stock on hand.

Write the changes to your mathematical model, defining any new variables. Distinguish physical stock, backlog, and net inventory. Implement the modified model and report the status, minimum cost, and end-of-week **net inventory** for g_2 in every week. Compare with part 3(b), identifying when g_2 is backlogged and the cost saving. Check that every backlog is zero and every final stock requirement is met at the end of week 12.

Explain why allowing delayed demand cannot increase the minimum cost.

Formulation changes and reasoning for 3(c):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work.

```
# Implement the backlog model here and compare its results with part 3(b).
goblet=Model(HiGHS.Optimizer)

@variable(goblet,x[G,T] >=0)
@variable(goblet,i[G,T]) #net inventory so it can now be negative
@variable(goblet,u[G,T]>=0) #unfilled demand at end of month in batches of 1000
@variable(goblet,l[G,T]>=0) #inventory left over for next month

@constraint(goblet,storage_limit[t in T],
    sum(l[g,t]*s[g] for g in G) <= S)

@constraint(goblet,elf_limit[t in T],
    sum(x[g,t]*w[g] for g in G) <= H)

@constraint(goblet,machine_limit[t in T],
    sum(x[g,t]*machine_time[g] for g in G) <= M)

@constraint(goblet,inventory[g in G, t in T], i[g,t]==l[g,t]-u[g,t])
@constraint(goblet, initial_inventory[g in G],
    x[g,1]+I0[g] == d[g,1]+i[g,1])
@constraint(goblet, production[g in G,t in 2:length(T)],
    x[g,t]+i[g,t-1] == d[g,t]+i[g,t])
@constraint(goblet, final[g in G],
    x[g,length(T)]+i[g,length(T)-1] >= d[g,length(T)]+IFinal[g])

@constraint(goblet, clear_backlog[g in G], u[g,length(T)]==0)

@objective(goblet, Min, sum((c[g]*x[g,t]+h[g]*l[g,t] for g in G, t in T)))
```

```

optimize!(goblet)
status = termination_status(goblet)
println("Termination status: ", status)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(goblet) || error("No feasible optimal solution is available")

minimum_cost_goblets = objective_value(goblet)
println("Minimum total cost: ", minimum_cost_goblets, " galleons")
solution= Dict{(g,t) => value(x[g,t]) for g in G, t in T)

#Print Product and amount Produced
println("")
for g in G, t in T
    println("Product: ",g," Week: ",
            t," Units Produced: ",solution[(g,t)], " Batches")
end

#Print Inventory at end of week
println("")
for g in G, t in T
    println("Product: ", g, " Week: ",
            t," Inventory: ", value(i[g,t]), " Batches")
end

#Print Unused Product at end of week
println("")
for g in G, t in T
    println("Product: ", g, " Week: ",
            t," Inventory: ", value(u[g,t]), " Batches")
end

#Print Inventory for G2 at end of week
println("")
for t in T
    println("Product: ", "g2", " Week: ",
            t," Inventory: ", value(i["g2",t]), " Batches")
end

#Print Stock Balances
println("")
for t in T
    stock= sum(i[g,t]*s[g] for g in G)
    println("Storage room Used on week",
            t, ": ", value(stock), " trays")
end

#Print Elf Use

```

```

println("")
for t in T
    elf= sum(x[g,t]*w[g] for g in G)
    println("Elf Work Hours Used",
            t, ": ", value(elf), " hours")
end

#Print Machine Use
println("")
for t in T
    mech= sum(x[g,t]*machine_time[g] for g in G)
    println("Machine Hours Used",t, ": ", value(mech), " hours")
end

```

Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT license
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full details
Using BLAS: blastrampoline
LP has 192 rows; 288 cols; 660 nonzeros
Coefficient ranges:

```

Matrix [1e+00, 1e+01]
Cost    [1e+01, 2e+02]
Bound   [0e+00, 0e+00]
RHS     [3e+00, 1e+03]

```

Presolving model

180 rows, 274 cols, 634 nonzeros 0s

112 rows, 206 cols, 645 nonzeros 0s

Dependent equations search running on 69 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

111 rows, 204 cols, 642 nonzeros 0s

Presolve reductions: rows 111(-81); columns 204(-84); nonzeros 642(-18)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	1.0334785695e+03	Pr: 74(12322)	0.0s
146	1.8184040752e+05	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

```

Model status      : Optimal
Simplex iterations: 146
Objective value    : 1.8184040752e+05
P-D objective error: 0.0000000000e+00
HiGHS run time    : 0.00
Termination status: OPTIMAL

```

Minimum total cost: 181840.40752351095 galleons

Product: g1 Week: 1 Units Produced: 0.0 Batches
Product: g1 Week: 2 Units Produced: 0.0 Batches
Product: g1 Week: 3 Units Produced: 10.0 Batches
Product: g1 Week: 4 Units Produced: 33.0 Batches
Product: g1 Week: 5 Units Produced: 19.0 Batches
Product: g1 Week: 6 Units Produced: 19.0 Batches
Product: g1 Week: 7 Units Produced: 23.0 Batches
Product: g1 Week: 8 Units Produced: 20.0 Batches
Product: g1 Week: 9 Units Produced: 29.0 Batches
Product: g1 Week: 10 Units Produced: 30.0 Batches
Product: g1 Week: 11 Units Produced: 28.0 Batches
Product: g1 Week: 12 Units Produced: 42.0 Batches
Product: g2 Week: 1 Units Produced: 0.0 Batches
Product: g2 Week: 2 Units Produced: 16.0 Batches
Product: g2 Week: 3 Units Produced: 23.0 Batches
Product: g2 Week: 4 Units Produced: 20.0 Batches
Product: g2 Week: 5 Units Produced: 11.0 Batches
Product: g2 Week: 6 Units Produced: 10.0 Batches
Product: g2 Week: 7 Units Produced: 12.0 Batches
Product: g2 Week: 8 Units Produced: 30.551724137931046 Batches
Product: g2 Week: 9 Units Produced: 24.448275862068954 Batches
Product: g2 Week: 10 Units Produced: 23.0 Batches
Product: g2 Week: 11 Units Produced: 30.0 Batches
Product: g2 Week: 12 Units Produced: 22.0 Batches
Product: g3 Week: 1 Units Produced: 18.0 Batches
Product: g3 Week: 2 Units Produced: 35.0 Batches
Product: g3 Week: 3 Units Produced: 12.705882352941183 Batches
Product: g3 Week: 4 Units Produced: 14.294117647058817 Batches
Product: g3 Week: 5 Units Produced: 9.0 Batches
Product: g3 Week: 6 Units Produced: 21.0 Batches
Product: g3 Week: 7 Units Produced: 23.0 Batches
Product: g3 Week: 8 Units Produced: 15.0 Batches
Product: g3 Week: 9 Units Produced: 10.0 Batches
Product: g3 Week: 10 Units Produced: -0.0 Batches
Product: g3 Week: 11 Units Produced: 13.0 Batches
Product: g3 Week: 12 Units Produced: 27.0 Batches
Product: g4 Week: 1 Units Produced: 16.0 Batches
Product: g4 Week: 2 Units Produced: 45.0 Batches
Product: g4 Week: 3 Units Produced: 24.0 Batches
Product: g4 Week: 4 Units Produced: 38.0 Batches
Product: g4 Week: 5 Units Produced: 41.0 Batches
Product: g4 Week: 6 Units Produced: 20.0 Batches
Product: g4 Week: 7 Units Produced: 15.799999999999998 Batches
Product: g4 Week: 8 Units Produced: 40.200000000000002 Batches
Product: g4 Week: 9 Units Produced: 28.0 Batches
Product: g4 Week: 10 Units Produced: 12.0 Batches

Product: g4 Week: 11 Units Produced: 30.0 Batches
 Product: g4 Week: 12 Units Produced: 47.0 Batches
 Product: g5 Week: 1 Units Produced: 23.0 Batches
 Product: g5 Week: 2 Units Produced: 20.0 Batches
 Product: g5 Week: 3 Units Produced: 21.470588235294116 Batches
 Product: g5 Week: 4 Units Produced: 16.529411764705884 Batches
 Product: g5 Week: 5 Units Produced: 20.454545454545453 Batches
 Product: g5 Week: 6 Units Produced: 17.545454545454547 Batches
 Product: g5 Week: 7 Units Produced: 47.600000000000016 Batches
 Product: g5 Week: 8 Units Produced: 7.736206896551705 Batches
 Product: g5 Week: 9 Units Produced: 31.413793103448278 Batches
 Product: g5 Week: 10 Units Produced: 25.25 Batches
 Product: g5 Week: 11 Units Produced: 0.0 Batches
 Product: g5 Week: 12 Units Produced: 0.0 Batches
 Product: g6 Week: 1 Units Produced: 12.0 Batches
 Product: g6 Week: 2 Units Produced: 7.111111111111111 Batches
 Product: g6 Week: 3 Units Produced: 30.88888888888889 Batches
 Product: g6 Week: 4 Units Produced: 19.0 Batches
 Product: g6 Week: 5 Units Produced: 18.0 Batches
 Product: g6 Week: 6 Units Produced: 35.0 Batches
 Product: g6 Week: 7 Units Produced: 0.0 Batches
 Product: g6 Week: 8 Units Produced: 28.0 Batches
 Product: g6 Week: 9 Units Produced: 12.0 Batches
 Product: g6 Week: 10 Units Produced: 34.0 Batches
 Product: g6 Week: 11 Units Produced: 36.75 Batches
 Product: g6 Week: 12 Units Produced: 13.25 Batches

Product: g1 Week: 1 Inventory: 30.0 Batches
 Product: g1 Week: 2 Inventory: 8.0 Batches
 Product: g1 Week: 3 Inventory: -0.0 Batches
 Product: g1 Week: 4 Inventory: -2.0 Batches
 Product: g1 Week: 5 Inventory: -0.0 Batches
 Product: g1 Week: 6 Inventory: -0.0 Batches
 Product: g1 Week: 7 Inventory: -0.0 Batches
 Product: g1 Week: 8 Inventory: -0.0 Batches
 Product: g1 Week: 9 Inventory: -0.0 Batches
 Product: g1 Week: 10 Inventory: -0.0 Batches
 Product: g1 Week: 11 Inventory: -0.0 Batches
 Product: g1 Week: 12 Inventory: 10.0 Batches
 Product: g2 Week: 1 Inventory: 3.0 Batches
 Product: g2 Week: 2 Inventory: -0.0 Batches
 Product: g2 Week: 3 Inventory: -0.0 Batches
 Product: g2 Week: 4 Inventory: -0.0 Batches
 Product: g2 Week: 5 Inventory: -0.0 Batches
 Product: g2 Week: 6 Inventory: -0.0 Batches
 Product: g2 Week: 7 Inventory: -0.0 Batches
 Product: g2 Week: 8 Inventory: -3.4482758620689538 Batches
 Product: g2 Week: 9 Inventory: -0.0 Batches

Product: g2 Week: 10 Inventory: -0.0 Batches
 Product: g2 Week: 11 Inventory: -0.0 Batches
 Product: g2 Week: 12 Inventory: 10.0 Batches
 Product: g3 Week: 1 Inventory: -0.0 Batches
 Product: g3 Week: 2 Inventory: -0.0 Batches
 Product: g3 Week: 3 Inventory: -4.294117647058817 Batches
 Product: g3 Week: 4 Inventory: -0.0 Batches
 Product: g3 Week: 5 Inventory: -0.0 Batches
 Product: g3 Week: 6 Inventory: -0.0 Batches
 Product: g3 Week: 7 Inventory: -0.0 Batches
 Product: g3 Week: 8 Inventory: -0.0 Batches
 Product: g3 Week: 9 Inventory: -0.0 Batches
 Product: g3 Week: 10 Inventory: -0.0 Batches
 Product: g3 Week: 11 Inventory: -0.0 Batches
 Product: g3 Week: 12 Inventory: 10.0 Batches
 Product: g4 Week: 1 Inventory: -0.0 Batches
 Product: g4 Week: 2 Inventory: -0.0 Batches
 Product: g4 Week: 3 Inventory: -0.0 Batches
 Product: g4 Week: 4 Inventory: -0.0 Batches
 Product: g4 Week: 5 Inventory: -0.0 Batches
 Product: g4 Week: 6 Inventory: -0.0 Batches
 Product: g4 Week: 7 Inventory: -3.2000000000000206 Batches
 Product: g4 Week: 8 Inventory: -0.0 Batches
 Product: g4 Week: 9 Inventory: -0.0 Batches
 Product: g4 Week: 10 Inventory: -0.0 Batches
 Product: g4 Week: 11 Inventory: -0.0 Batches
 Product: g4 Week: 12 Inventory: 10.0 Batches
 Product: g5 Week: 1 Inventory: -0.0 Batches
 Product: g5 Week: 2 Inventory: -0.0 Batches
 Product: g5 Week: 3 Inventory: -1.529411764705884 Batches
 Product: g5 Week: 4 Inventory: -0.0 Batches
 Product: g5 Week: 5 Inventory: 10.454545454545453 Batches
 Product: g5 Week: 6 Inventory: 6.0 Batches
 Product: g5 Week: 7 Inventory: 35.600000000000016 Batches
 Product: g5 Week: 8 Inventory: 13.336206896551722 Batches
 Product: g5 Week: 9 Inventory: 16.75 Batches
 Product: g5 Week: 10 Inventory: 35.0 Batches
 Product: g5 Week: 11 Inventory: 20.0 Batches
 Product: g5 Week: 12 Inventory: 10.0 Batches
 Product: g6 Week: 1 Inventory: -0.0 Batches
 Product: g6 Week: 2 Inventory: -10.88888888888889 Batches
 Product: g6 Week: 3 Inventory: -0.0 Batches
 Product: g6 Week: 4 Inventory: -0.0 Batches
 Product: g6 Week: 5 Inventory: -0.0 Batches
 Product: g6 Week: 6 Inventory: -0.0 Batches
 Product: g6 Week: 7 Inventory: -0.0 Batches
 Product: g6 Week: 8 Inventory: -0.0 Batches
 Product: g6 Week: 9 Inventory: -0.0 Batches

Product: g6 Week: 10 Inventory: 4.0 Batches
Product: g6 Week: 11 Inventory: 19.75 Batches
Product: g6 Week: 12 Inventory: 10.0 Batches

Product: g1 Week: 1 Inventory: 0.0 Batches
Product: g1 Week: 2 Inventory: 0.0 Batches
Product: g1 Week: 3 Inventory: 0.0 Batches
Product: g1 Week: 4 Inventory: 2.0 Batches
Product: g1 Week: 5 Inventory: 0.0 Batches
Product: g1 Week: 6 Inventory: 0.0 Batches
Product: g1 Week: 7 Inventory: 0.0 Batches
Product: g1 Week: 8 Inventory: 0.0 Batches
Product: g1 Week: 9 Inventory: 0.0 Batches
Product: g1 Week: 10 Inventory: 0.0 Batches
Product: g1 Week: 11 Inventory: 0.0 Batches
Product: g1 Week: 12 Inventory: 0.0 Batches
Product: g2 Week: 1 Inventory: 0.0 Batches
Product: g2 Week: 2 Inventory: 0.0 Batches
Product: g2 Week: 3 Inventory: 0.0 Batches
Product: g2 Week: 4 Inventory: 0.0 Batches
Product: g2 Week: 5 Inventory: 0.0 Batches
Product: g2 Week: 6 Inventory: 0.0 Batches
Product: g2 Week: 7 Inventory: 0.0 Batches
Product: g2 Week: 8 Inventory: 3.4482758620689538 Batches
Product: g2 Week: 9 Inventory: 0.0 Batches
Product: g2 Week: 10 Inventory: 0.0 Batches
Product: g2 Week: 11 Inventory: 0.0 Batches
Product: g2 Week: 12 Inventory: 0.0 Batches
Product: g3 Week: 1 Inventory: 0.0 Batches
Product: g3 Week: 2 Inventory: 0.0 Batches
Product: g3 Week: 3 Inventory: 4.294117647058817 Batches
Product: g3 Week: 4 Inventory: 0.0 Batches
Product: g3 Week: 5 Inventory: 0.0 Batches
Product: g3 Week: 6 Inventory: 0.0 Batches
Product: g3 Week: 7 Inventory: 0.0 Batches
Product: g3 Week: 8 Inventory: 0.0 Batches
Product: g3 Week: 9 Inventory: 0.0 Batches
Product: g3 Week: 10 Inventory: 0.0 Batches
Product: g3 Week: 11 Inventory: 0.0 Batches
Product: g3 Week: 12 Inventory: 0.0 Batches
Product: g4 Week: 1 Inventory: 0.0 Batches
Product: g4 Week: 2 Inventory: 0.0 Batches
Product: g4 Week: 3 Inventory: 0.0 Batches
Product: g4 Week: 4 Inventory: 0.0 Batches
Product: g4 Week: 5 Inventory: 0.0 Batches
Product: g4 Week: 6 Inventory: 0.0 Batches
Product: g4 Week: 7 Inventory: 3.2000000000000206 Batches
Product: g4 Week: 8 Inventory: 0.0 Batches

Product: g4 Week: 9 Inventory: 0.0 Batches
 Product: g4 Week: 10 Inventory: 0.0 Batches
 Product: g4 Week: 11 Inventory: 0.0 Batches
 Product: g4 Week: 12 Inventory: 0.0 Batches
 Product: g5 Week: 1 Inventory: 0.0 Batches
 Product: g5 Week: 2 Inventory: 0.0 Batches
 Product: g5 Week: 3 Inventory: 1.529411764705884 Batches
 Product: g5 Week: 4 Inventory: 0.0 Batches
 Product: g5 Week: 5 Inventory: 0.0 Batches
 Product: g5 Week: 6 Inventory: 0.0 Batches
 Product: g5 Week: 7 Inventory: 0.0 Batches
 Product: g5 Week: 8 Inventory: 0.0 Batches
 Product: g5 Week: 9 Inventory: 0.0 Batches
 Product: g5 Week: 10 Inventory: 0.0 Batches
 Product: g5 Week: 11 Inventory: 0.0 Batches
 Product: g5 Week: 12 Inventory: 0.0 Batches
 Product: g6 Week: 1 Inventory: 0.0 Batches
 Product: g6 Week: 2 Inventory: 10.88888888888889 Batches
 Product: g6 Week: 3 Inventory: 0.0 Batches
 Product: g6 Week: 4 Inventory: 0.0 Batches
 Product: g6 Week: 5 Inventory: 0.0 Batches
 Product: g6 Week: 6 Inventory: 0.0 Batches
 Product: g6 Week: 7 Inventory: 0.0 Batches
 Product: g6 Week: 8 Inventory: 0.0 Batches
 Product: g6 Week: 9 Inventory: 0.0 Batches
 Product: g6 Week: 10 Inventory: 0.0 Batches
 Product: g6 Week: 11 Inventory: 0.0 Batches
 Product: g6 Week: 12 Inventory: 0.0 Batches

Product: g2 Week: 1 Inventory: 3.0 Batches
 Product: g2 Week: 2 Inventory: -0.0 Batches
 Product: g2 Week: 3 Inventory: -0.0 Batches
 Product: g2 Week: 4 Inventory: -0.0 Batches
 Product: g2 Week: 5 Inventory: -0.0 Batches
 Product: g2 Week: 6 Inventory: -0.0 Batches
 Product: g2 Week: 7 Inventory: -0.0 Batches
 Product: g2 Week: 8 Inventory: -3.4482758620689538 Batches
 Product: g2 Week: 9 Inventory: -0.0 Batches
 Product: g2 Week: 10 Inventory: -0.0 Batches
 Product: g2 Week: 11 Inventory: -0.0 Batches
 Product: g2 Week: 12 Inventory: 10.0 Batches

Storage room Used on week1: 135.0 trays
 Storage room Used on week2: -66.0 trays
 Storage room Used on week3: -27.588235294117624 trays
 Storage room Used on week4: -8.0 trays
 Storage room Used on week5: 41.81818181818181 trays
 Storage room Used on week6: 24.0 trays

Storage room Used on week7: 123.19999999999993 trays
 Storage room Used on week8: 36.10344827586212 trays
 Storage room Used on week9: 67.0 trays
 Storage room Used on week10: 176.0 trays
 Storage room Used on week11: 257.75 trays
 Storage room Used on week12: 330.0 trays

Elf Work Hours Used1: 226.0 hours
 Elf Work Hours Used2: 351.44444444444446 hours
 Elf Work Hours Used3: 394.55555555555554 hours
 Elf Work Hours Used4: 420.0 hours
 Elf Work Hours Used5: 352.8181818181818 hours
 Elf Work Hours Used6: 400.1818181818182 hours
 Elf Work Hours Used7: 396.0 hours
 Elf Work Hours Used8: 420.0 hours
 Elf Work Hours Used9: 420.0 hours
 Elf Work Hours Used10: 420.0 hours
 Elf Work Hours Used11: 420.0 hours
 Elf Work Hours Used12: 420.0 hours

Machine Hours Used1: 561.0 hours
 Machine Hours Used2: 800.0 hours
 Machine Hours Used3: 800.0 hours
 Machine Hours Used4: 800.0 hours
 Machine Hours Used5: 800.0 hours
 Machine Hours Used6: 800.0 hours
 Machine Hours Used7: 800.0 hours
 Machine Hours Used8: 789.25 hours
 Machine Hours Used9: 800.0 hours
 Machine Hours Used10: 762.75 hours
 Machine Hours Used11: 708.75 hours
 Machine Hours Used12: 709.25 hours

Results and interpretation for 3(c):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work.

New Decision Variables

$l_{g,t}$ Left over inventory (batch 1000 goblets) $l_{g,t} \geq 0$

$u_{g,t}$ Unfilled inventory (batch 1000 goblets) $u_{g,t} \geq 0$

$i_{g,t}$ net inventory (batch 1000 goblets)

$$i_t = l_t - u_t.$$

The minimum cost for this goblet manufacturing scenario is 181840 galleons. The introduction of backlogging reduces the overall minimum cost because it relaxes the constraints and thus increases

the feasible region size. This can only result in a reduced minimum or a equivalent result, it cannot increase the minimum cost. Compared to the no backlogging version goblet 2 gets backlogged on week 8 instead of producing exactly the minimum to meet demand.

Collaboration and LLM-use statement — required

Complete this statement as the **very last part of your homework**. Be specific enough that a reader can understand what work you did yourself and how others or LLMs helped you learn.

- **People:** Give the name of every person you collaborated with, identify the problem/subpart you discussed, and explain the nature of the collaboration. If you worked alone, explicitly state that you did not collaborate with anyone.
- **LLMs/tools:** Name every LLM or AI assistant you used and identify the model/version as precisely as available. If the interface did not show a model/version, say so. If you used none, explicitly state that you did not use an LLM or AI assistant.
- **Specific assistance and learning:** For each tool, identify the problem/subpart, describe what you asked it to help with, what suggestions you used or rejected, and how the interaction helped you understand the mathematics or implementation. Explain how you checked the suggestions. “I used ChatGPT for help” is not sufficiently specific.

Disclosure does not replace the requirement that your submitted formulation, implementation, and explanation be your own work and that you understand them.

People and collaboration:

[Names, problem/subparts, and what you discussed; or explicitly state that you worked alone.]

I worked alone on this assignment.

LLMs/tools and models:

[Tool names and models/versions, or explicitly state that you used none.]

Google Gemini 3.5 Flash Lite

How I used the assistance, what I learned, and how I checked it:

[Specific details for each tool and relevant problem/subpart; state not applicable if you used none.]

I used this to assist me on problem 2 to help understand the data frame formatting and why I needed to use value. instead of value. The formatting was checked by having it provide the documentaion for data frame in julia and the value. was needed becasue it was funcitoning on an array not a single value. The other portion of the asssignement that was assisted by LLM was in debugging. Some times due to how Jump is implimented the errors wont display what line of the code the error stems from. When these situations occur I supply my code and the error and ask for what line the error is on. I then manually debug from there to ensure that I understand what error occured rather than having it tell me what mistake I made.